Reviewer Pack — Coxeter-Dynkin Diagram Symmetry and Boolean Function Monoton…

Entry 8120299b4b6a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 16:47:01 UTC

Coxeter-Dynkin Diagram Symmetry and Boolean Function Monotonicity

Entry ID: `8120299b4b6a` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 16:47:01 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Group Theory **Field B** (complexity object): Boolean Function Complexity

Statement:

> For any boolean function f, the number of symmetry classes of its Coxeter-Dynkin diagram (under the action of the symmetric group on vertices) is bounded by a polynomial in the degree of f, and this bound holds for all functions monotonic to f.

Rationale (proposer's reasoning):

> Coxeter groups have been used to study symmetry in geometric problems. If this relationship between symmetry and complexity holds, it could provide new insights into the structure of boolean functions and potentially lead to more efficient algorithms for solving problems related to boolean function monotonicity.

Taxonomy category: `FOURIER_ANALYTIC` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f3ef83ebd136ea88`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The number of symmetry classes in the Coxeter-Dynkin diagram for any boolean function is bounded by a polynomial P(n), where n is the degree of the function, and this bound applies to all functions monotonic to f.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"Coxeter-Dynkin Diagram symmetry" AND "Boolean function monotonicity"` - `"combinatorial group theory" AND "Boolean function complexity" AND "polynomial bound"` - `"symmetric group action" ON "vertices of Coxeter-Dynkin diagram" AND "boolean function degree"`

Top relevant hits considered: - [http://arxiv.org/abs/0709.1296v2] Twisted Symmetric Group Actions - [http://arxiv.org/abs/2001.07613v1] Cut-Based Graph Learning Networks to Discover Compositional Structure of Sequential Video Data - [http://arxiv.org/abs/2403.10838v1] Two-step Automated Cybercrime Coded Word Detection using Multi-level Representation Learning - [s2:03c1047703a79bbf873ef70a22f76b6df82a095e] Theory and applications of models of computation : 4th International Conference, TAMC 2007, Shanghai, China, May 22-25, - [s2:03680ecd0cd8c9d658a05bbd53c822d6a1c11cda] Approximation, randomization and combinatorial optimization : algorithms and techniques : 15th International Workshop, A

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.4s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def is_monotonic(f):
    n = len(f)
    for i in range(n):
        if any(f[i | (1 << j)] < f[i] for j in range(n)):
            return False
    return True

def generate_boolean_function(n):
    return [random.choice([0, 1]) for _ in range(2**n)]

def construct_coxeter_dynkin_diagram(f):
    n = len(f)
    diagram = {}
    for i in range(2**n):
        for j in range(n):
            if f[i | (1 << j)] < f[i]:
                diagram[(i, i | (1 << j))] = 1
    return diagram

def count_symmetry_classes(diagram):
```

```
    n = len(diagram)
    visited = [False] * n
    classes = 0

    def dfs(node):
        if not visited[node]:
            visited[node] = True
            for neighbor in range(n):
                if (node, neighbor) in diagram or (neighbor, node) in diagram:
                    dfs(neighbor)

    for i in range(n):
        if not visited[i]:
            classes += 1
            dfs(i)

    return classes

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    f = generate_boolean_function(n)

    if not is_monotonic(f):
        return {
            "metric_name": "symmetry_classes",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "function_not_monotonic"
        }

    diagram = construct_coxeter_dynkin_diagram(f)
    symmetry_classes = count_symmetry_classes(diagram)

    return {
        "metric_name": "symmetry_classes",
        "metric_value": symmetry_classes,
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        support_fraction = 1.0
```

```
        print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        counterexample = next(result["counterexample"] for result in results if not result["conjec
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TRIAL: {'metric_name': 'symmetry_classes', 'metric_value': None, 'instances_tested': 1, 'conjecture_

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_39f89856.py", line 87, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_39f89856.py", line 61, in run_trial
    if not is_monotonic(f):
           ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_39f89856.py", line 21, in is_monotonic
    if any(f[i | (1 << j)] < f[i] for j in range(n)):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_39f89856.py", line 21, in <genexpr>
    if any(f[i | (1 << j)] < f[i] for j in range(n)):
           ~^^^^^^^^^^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data that could confirm or falsify the conjecture. | next: Review and debug the test code to ensure it can complete without crashing, then rerun the test with a known monotonic function to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14647
2	preregistration	ollama_remote	glm4:latest	0	0	9571
3	novelty	ollama_remote	glm4:latest	0	0	8551
4	novelty	ollama_remote	glm4:latest	0	0	9519

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13240
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9925
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8900
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8320
9	judge	ollama_remote	glm4:latest	0	0	11530

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 94204 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/8120299b4b6a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8120299b4b6a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/8120299b4b6a.tar.gz` (if generated)